



Visualizing a PyTorch Model

by **Adrian Tam** on April 8, 2023 in **Deep Learning with PyTorch** 2



Post

PyTorch is a deep learning library. You can build very sophisticated deep learning models with PyTorch. However, there are times you want to have a graphical representation of your model architecture. In this post, you will learn:

- How to save your PyTorch model in an exchange format
- How to use Netron to create a graphical representation.

Kick-start your project with my book [Deep Learning with PyTorch](#). It provides **self-study tutorials** with **working code**.

Let's get started.



Visualizing a PyTorch Model
Photo by Ken Cheung. Some rights reserved.

Overview

This post is divided into two parts; they are

- Why Graphical Representation of a PyTorch Model is Hard
- How to Use Netron to Create a Model Graph

Why Graphical Representation of a PyTorch Model is Hard

PyTorch is a very flexible library for deep learning. Strictly speaking, it never mandates how you should build your model as long as it works like a function that can transform an input tensor into output tensor. This is a problem: With a model, you never know how it works unless you follow the input tensor and collect the trace until you get the output tensor. Therefore it is not trivial to convert a PyTorch model into a picture.

There are multiple libraries to solve this problem. But in general, there are only two ways to solve it: You can follow a tensor on the forward pass and see what operation (i.e., layers) are applied, or

follow a tensor on the backward pass and see how the gradient propagated to the input. You can only find clues about the structure inside a model in this manner.

Want to Get Started With Deep Learning with PyTorch?

Take my free email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

How to Use Netron to Create a Model Graph

When you save a PyTorch model, you are saving its states. You can get the model states using `model.state_dict()`. While the weight tensors have names so it helps you to restore them to a model, you do not have the clues on how the weights are connected to each other. The only way you can connect tensors and find out their relationship is to get the tensor gradient: When you run a model and get an output, the calculations involved, including the dependencies to other tensors, are remembered by each intermediate tensors so automatic differentiation can be performed.

Indeed, if you want to know the algorithm behind a PyTorch model, this is also the way to go. There are only a few tools to create graphics from a PyTorch model. In below, you will learn about the tool Netron. It is a “deep learning model viewer”. It is a software that you can install and run on macOS, Linux, and Windows. You can visit the page below and download the software for your platform:

- <https://github.com/lutzroeder/netron/releases>

There is also an [online version](#) available, that you can see your model by uploading a model file.

Netron cannot visualize a PyTorch model from the saved states because there's not enough clues to tell about the structure of the model. However, PyTorch allows you to convert the model to an exchange format, ONNX, that Netron can understand.

Let's start with an example. In below you created a simple model to classify the iris dataset. It is a classification problem with three classes. Therefore the model should output a vector of three values. A complete code that you would create for this problem is as follows, which the dataset is obtained from scikit-learn:

```
1 import torch
```

```

2 import torch.nn as nn
3 import torch.optim as optim
4 from sklearn.datasets import load_iris
5 from sklearn.model_selection import train_test_split
6
7 data = load_iris()
8 X = data['data']
9 y = data['target']
10 X = torch.tensor(X, dtype=torch.float32)
11 y = torch.tensor(y, dtype=torch.long)
12
13 # split
14 X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)
15
16 class IrisModel(nn.Module):
17     def __init__(self):
18         super().__init__()
19         self.hidden = nn.Linear(4, 8)
20         self.act = nn.ReLU()
21         self.output = nn.Linear(8, 3)
22
23     def forward(self, x):
24         x = self.act(self.hidden(x))
25         x = self.output(x)
26         return x
27
28 # loss metric and optimizer
29 model = IrisModel()
30 loss_fn = nn.CrossEntropyLoss()
31 optimizer = optim.Adam(model.parameters(), lr=0.001)
32
33 # prepare model and training parameters
34 n_epochs = 100
35 batch_size = 10
36 batch_start = torch.arange(0, len(X_train), batch_size)
37
38 # training loop
39 for epoch in range(n_epochs):
40     for start in batch_start:
41         # take a batch
42         X_batch = X_train[start:start+batch_size]
43         y_batch = y_train[start:start+batch_size]
44         # forward pass
45         y_pred = model(X_batch)
46         loss = loss_fn(y_pred, y_batch)
47         # backward pass
48         optimizer.zero_grad()
49         loss.backward()
50         # update weights
51         optimizer.step()
52
53 # validating model
54 y_pred = model(X_test)
55 acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
56 acc = float(acc)*100
57 print("Model accuracy: %.2f%%" % acc)

```

Running the above produces the following, for example:

```
1 Model accuracy: 97.78%
```

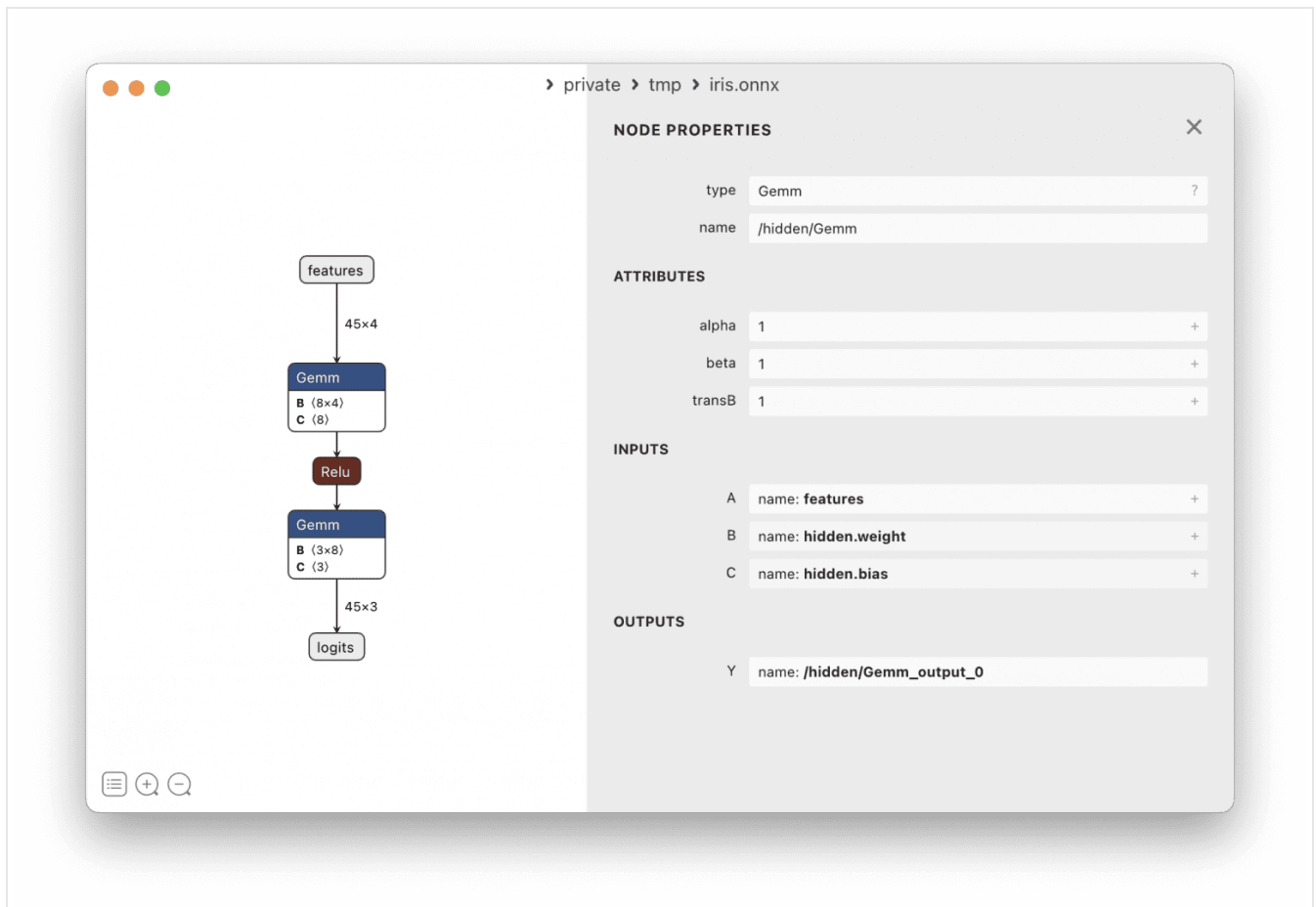
So you know that model is a PyTorch model that can take a tensor and return a tensor. You can **convert** this model into ONNX format using the `torch.onnx.export()` function:

```
1 torch.onnx.export(model, X_test, 'iris.onnx', input_names=["features"], output_names=["logits"]
```

Running this will create a file `iris.onnx` at the local directory. You need to provide a **sample tensor** that works with the model as input (`X_test` in the example above). This is because during the conversion, it needs to follow this sample tensor to understand what operations shall be applied so you can convert the algorithm step by step into ONNX format. Every weight in a PyTorch model is a tensor and there is a name assigned to them. But the input and output tensors are not named usually, hence you need to provide a name to them when you run `export()`. These names should be provided as a list of strings because in general, a model can take multiple tensors and return multiple tensors.

Usually you should run `export()` after the training loop. It is because the ONNX model created contains a complete model that you can run without the PyTorch library. You want to save the optimized weight to it. However, for the purpose of visualizing a model in Netron, the quality of the model is not a concern. You can run `export()` as soon as the PyTorch model is created.

After you launched Netron, you can open the saved ONNX file. In this example, you should see the following screen:



It shows how the input tensor connected to the output tensor via different operations in the deep learning model. The name of the input and output tensors you provided to the `export()` function is used in the visualization. Clicking on a box will give you more details about that particular tensor or

operation. The names of operations as you can see in Netron may be different from what you call them in PyTorch, however. In the screen above, you see the `nn.Linear()` layer becomes “Gemm”, which stands for “general matrix multiply” operation. You can even examine the weights on a layer with Netron by a few points and clicks.

If you would like to keep a copy of this visualization, you can export it into PNG format in Netron.

Further Readings

Netron is an open source project which you can find its source code in Github:

- <https://github.com/lutzroeder/netron>

An online version of Netron is available below:

- <https://netron.app/>

Another library for visualization is torchviz, but unlike the example you saw above, it traces the model from a backward pass:

- <https://github.com/szagoruyko/pytorchviz>

Summary

In this post, you learned how to visualize a model. In particular, you learned:

- Why visualizing a PyTorch model is difficult
- How to convert a PyTorch model into ONNX format
- How to use Netron to visualize a ONNX model

Get Started on Deep Learning with PyTorch!

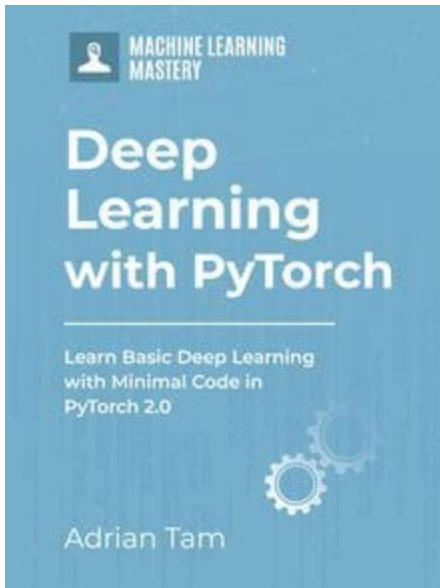
Learn how to build deep learning models

...using the newly released PyTorch 2.0 library

Discover how in my new Ebook:
Deep Learning with PyTorch

It provides **self-study tutorials** with **hundreds of working code** to turn you from a novice to expert. It equips you with

tensor operation, training, evaluation, hyperparameter optimization, and much more...



Kick-start your deep learning journey with hands-on exercises

SEE WHAT'S INSIDE

[Share](#) [Share](#) [Post](#)

More On This Topic



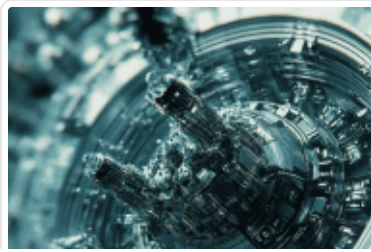
Understand Model Behavior During Training by...



Visualizing the vanishing gradient problem



Revealing the Invisible: Visualizing Missing Values...



About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.

[View all posts by Adrian Tam](#) →

< Managing a PyTorch Training Process with Checkpoints and Early Stopping

Building a Convolutional Neural Network in PyTorch >

2 Responses to *Visualizing a PyTorch Model*



Trương Quốc Quân March 10, 2023 at 1:50 pm #

REPLY ↩

Sweet! Would that work with TensorFlow and Keras? I've been using StellarGraph, a Graph Neural Network Library built upon TensorFlow. Tensorflow also supports exporting models to ONNXs.



James Carmichael March 11, 2023 at 7:56 am #

REPLY ↩

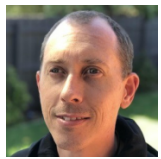
Hi Truong...Absolutely! The following is a great starting point using TensorFlow and Keras.

<https://machinelearningmastery.com/tensorflow-tutorial-deep-learning-with-tf-keras/>

Leave a Reply

Name (required)

Email (will not be published) (required)

[SUBMIT COMMENT](#)**Welcome!***I'm Jason Brownlee PhD*and I **help developers** get results with **machine learning**.[Read more](#)**Never miss a tutorial:****Picked for you:**[PyTorch Tutorial: How to Develop Deep Learning Models with Python](#)



LSTM for Time Series Prediction in PyTorch



Deep Learning with PyTorch (9-Day Mini-Course)



Calculating Derivatives in PyTorch



Develop Your First Neural Network with PyTorch, Step by Step

Loving the Tutorials?

The Deep Learning with PyTorch EBook
is where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE



Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. [Visit our corporate website](#) to learn more about our mission and team.



[PRIVACY](#) | [DISCLAIMER](#) | [TERMS](#) | [CONTACT](#) | [SITEMAP](#) | [ADVERTISE WITH US](#)

© 2026 Guiding Tech Media All Rights Reserved